

APS Backend (aps-be)

Backend implementation for the Apartment Booking System. This repository contains two API interfaces over the same APS domain model and PostgreSQL database:

- REST API, served on `http://{BASE_URL}:3001/api`
- GraphQL API, served on `http://{BASE_URL}:3002/graphql`

The project is designed to support a controlled REST versus GraphQL performance comparison. Both APIs use the same database, authentication model, validation rules, business logic, and Docker resource limits so the benchmark can focus on API architecture and request shape.

Project Scope

This repository includes:

- TypeScript REST API using Express.
- TypeScript GraphQL API using Apollo Server.
- Shared PostgreSQL access and domain services.
- JWT authentication with an OTP flow that can be disabled for load testing.
- Docker Compose configuration for running both API containers.
- k6 scenarios for paired REST and GraphQL load tests.
- Result comparison and resource monitoring scripts.
- Detailed API and testing documentation.

The database schema and seed data live in the separate `aps-db` project. Start `aps-db` before running the backend in Docker.

Documentation Map

Document	Purpose
<code>src/rest/REST.md</code>	REST endpoint reference, request examples, authentication behavior, and error format
<code>src/graphql/GRAPH.md</code>	GraphQL endpoint reference, queries, mutations, authentication behavior, and error format
<code>tests/docs/load-testing.md</code>	Benchmark design, scenarios, metrics, controls, result files, and analysis guidance
<code>tests/docs/testing-guide.md</code>	Step-by-step instructions for inspecting provided results or running a fresh benchmark
<code>tests/results.zip</code>	Provided benchmark output archive for quick analysis without rerunning the full suite

Use this README as the starting point. Use the documents above for complete operational and API details.

Repository Structure

aps-be/	
docker/	Dockerfiles for REST and GraphQL services
scripts/	Result comparison and Docker resource monitoring
scripts	
src/	
core/	Shared domain logic, repositories, middleware,
and types	
db/	PostgreSQL client setup
rest/	Express REST API and REST documentation
graphql/	Apollo GraphQL API, schema, resolvers, and
GraphQL documentation	
utils/	Shared utility code
tests/	
docs/	Benchmark and testing documentation
k6/	k6 load-test configuration, helpers, and
scenarios	
results.zip	Provided benchmark result archive
docker-compose.yml	REST and GraphQL API containers
package.json	Node.js scripts and dependencies

Prerequisites

- Node.js 20 or newer.
- npm.
- Docker and Docker Compose.
- PostgreSQL database from the [aps-db](#) project.
- k6, required only for running fresh load tests.
- [unzip](#), required only for extracting the provided result archive.
- [jq](#), optional for inspecting JSON result files.

Environment

The backend reads configuration from `.env`. The current local setup expects the database container created by [aps-db](#):

```
DATABASE_URL=postgresql://apsadmin:adminpassword@aps-
postgres:5432/apartment_booking
DB_HOST=aps-postgres
DB_PORT=5432
DB_USER=apsadmin
DB_PASSWORD=adminpassword
DB_NAME=apartment_booking

REST_PORT=3001
GRAPHQL_PORT=3002
JWT_SECRET=replace-with-a-local-secret
JWT_EXPIRES_IN=604800
NODE_ENV=development
LOG_LEVEL=info
```

```
LOG_QUERIES=false
CORS_ORIGIN=http://localhost:3000

GRAPHQL_URL=http://localhost:3002/graphql
REST_URL=http://localhost:3001/api

DISABLE_OTP=true
BYPASS_OTP_NUMBER=+6281234567890
BYPASS_OTP_CODE=0000
```

For load testing, `DISABLE_OTP=true` is required because the k6 helpers authenticate by phone number and expect a JWT token directly from login. For normal OTP behavior, set `DISABLE_OTP=false`.

Do not reuse local development secrets or OTP provider keys in a production environment.

Quick Start with Docker

Start the database first from the sibling `aps-db` project:

```
cd ../aps-db
docker compose up -d
```

Then start the backend:

```
cd ../aps-be
npm install
docker compose up -d --build
docker compose ps
```

Expected backend containers:

```
aps-rest-api
aps-graphql-api
```

Verify the REST API:

```
curl http://localhost:3001/health
```

Verify the GraphQL API:

```
curl -X POST http://localhost:3002/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __typename }" }'
```

Local Development Without API Containers

Run the database with `aps-db`, then install dependencies and start both API servers:

```
npm install
npm run dev
```

Run one API at a time when needed:

```
npm run dev:rest
npm run dev:graphql
```

Useful development commands:

```
npm run build
npm test
npm run test:parity
```

Authentication for Testing

REST login:

```
curl -X POST http://localhost:3001/api/auth/login \
-H "Content-Type: application/json" \
-d '{ "phoneNumber": "+6281234567892" }'
```

GraphQL login:

```
curl -X POST http://localhost:3002/graphql \
-H "Content-Type: application/json" \
-d '{
  "query": "mutation Login($input: LoginInput!) { login(input: $input) {
token user { id fullName role } sessionId expires } }",
  "variables": { "input": { "phoneNumber": "+6281234567892" } }
}'
```

When `DISABLE_OTP=true`, both responses should include a `token`. When OTP is enabled, login starts an OTP session and returns `sessionId` and `expires` instead.

API References

REST:

- Base URL: `http://localhost:3001/api`
- Full documentation: `src/rest/REST.md`
- Postman collection: `src/rest/APS REST API.postman_collection.json`

GraphQL:

- Endpoint: `http://localhost:3002/graphql`
- Full documentation: `src/graphql/GRAPH.md`
- Schema: `src/graphql/schema/schema.graphql`
- Postman collection: `src/graphql/APS GraphQL API.postman_collection.json`

Load Testing and Research Workflow

There are two supported workflows.

Analyze the Provided Results

Use this workflow to inspect the supplied benchmark output without starting Docker containers or running the full benchmark:

```
mkdir -p tests/results
unzip -o tests/results.zip -d tests/results
node scripts/compare-results.js
```

The comparison script reads `tests/results/rest` and `tests/results/graphql`, then reports latency, throughput, error rate, resource usage, and transferred data by scenario.

Run a Fresh Benchmark

Start `aps-db`, start both backend API containers, confirm `DISABLE_OTP=true`, then run:

```
./tests/k6/run-all.sh
node scripts/compare-results.js
```

The benchmark runs six paired REST and GraphQL scenarios:

Scenario	Name	Main comparison
01	Simple list	Public facility list retrieval
02	List with relations	User bookings with nested user and facility data
03	Filtered slots	Facility availability lookup
04	Generic nested query	Client-composed REST requests versus a composed GraphQL query

Scenario	Name	Main comparison
05	Optimized nested query	Dedicated dashboard endpoint versus dedicated dashboard resolver
06	Booking creation	Write performance, validation, conflicts, and cleanup

The full run usually takes more than one hour. For methodology, metrics, controls, result structure, and troubleshooting, read [tests/docs/load-testing.md](#) and [tests/docs/testing-guide.md](#).

Result Files

Fresh benchmark output is written under:

```
tests/results/  
  rest/  
    01-simple-list-raw.json  
    01-simple-list-summary.json  
    01-simple-list-resources.csv  
    ...  
  graphql/  
    01-simple-list-raw.json  
    01-simple-list-summary.json  
    01-simple-list-resources.csv  
    ...
```

Keep raw JSON and CSV files when using the benchmark in a research report. They provide the evidence for latency, throughput, resource, and network-transfer analysis.

Troubleshooting

If API containers cannot connect to the database, confirm that [aps-db](#) is running and that the external Docker network [aps-db_aps-network](#) exists.

If login returns [sessionId](#) instead of [token](#) during k6 tests, set [DISABLE_OTP=true](#) and recreate the API containers:

```
docker compose up -d --force-recreate rest-api graphql-api
```

If [node scripts/compare-results.js](#) reports missing files, extract [tests/results.zip](#) into [tests/results](#) or run the full benchmark first.